

■ Amazon ML Challenge 2026

Problem Statement & Team Understanding Document

1. What is the Problem?

We are solving an **Entity Resolution / Entity Matching** problem.

Amazon has business information coming from multiple data sources. The same real-world business may appear differently across sources because of:

- Spelling mistakes and typos
- Different formatting
- Abbreviations
- Missing information
- Different address formats
- Multilingual names
- Extra or missing words

Our job is to identify which **Source 2** and **Source 3** entities correspond to each **Source 1** entity.

Example from the actual dataset

Source 1: Payne Enterprises — 3315 Fremont Street, Peoria, IL, US

Possible matching records:

- PAYNE-ENRTPRMISES — 3315 FREMONT SAINT, PEORIA, IL
- Payne Énterprises — 3315 FREMONT ST, PEORIA, IL
- Payne Enterprises LLC — Fremont St, Peoria, Illinois
- Payne Etrepndiels — 3315 Fremont St, Peoria, Illinois

All of these are considered matches in our training ground truth — so **exact string matching will not work**.

2. What Data Do We Have?

Repository structure:

```
dataset/  
|  
|-- train/  
|   |-- train_source1.tsv  
|   |-- train_source2.tsv  
|   |-- train_source3.tsv  
|   |-- train_ground_truth.tsv  
|  
|-- test/  
|   |-- test_source1.tsv  
|   |-- test_source2.tsv  
|   |-- test_source3.tsv
```

Training data

File	Rows
train_source1	2,206,821
train_source2	5,034,616
train_source3	5,285,603
train_ground_truth	2,206,821

Test data

File	Rows
test_source1	1,732,544
test_source2	4,887,273
test_source3	5,082,316

Source files contain: entity_id, business_name, business_address, country.

Ground truth contains: source1_entity_id, matched_entity_ids.

3. Very Important: This is NOT One-to-One Matching

One Source 1 business can have **multiple** matching entities across Source 2 and Source 3.

From our actual ground truth — minimum matches per Source 1 entity: **1**, maximum: **11**, average: **3.52**.

Number of matches	Count of Source 1 entities
1	242,404
2	375,212
3	530,841
4	484,115
5	321,957
6	164,868
7	63,968
8	18,680
9	4,205
10	534
11	37

So our model cannot simply predict one match per Source 1 entity.

4. Why Is This Difficult?

The datasets contain noisy and multilingual information.

Name variations

- Payne Enterprises
- Payne Énterprises
- Payne Enterpires
- PAYNE-ENRTPRMISES
- Payne Enterprises LLC
- Payne Etrepndiels

Address variations

- 3315 Fremont Street, Peoria, IL
- 3315 FREMONT ST, PEORIA, IL
- Fremont St, Peoria, Illinois
- 3315Fremont Street, Peoria, Illinois

Multilingual variations

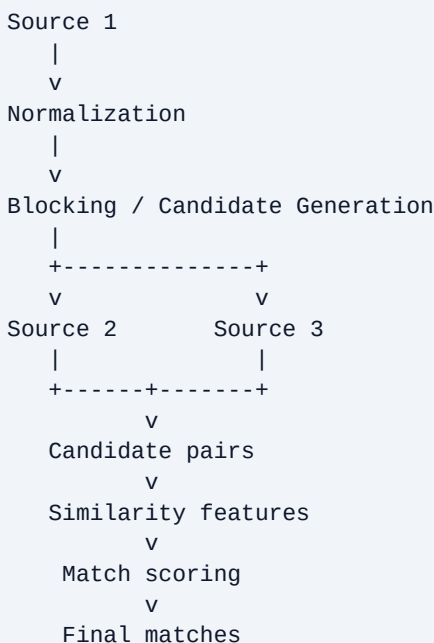
The same entity can appear in English and another script — e.g. "Raj Investments LLP" may also appear in a different script or in a regional-language representation.

So our solution needs to handle **noisy + multilingual** entity information.

5. The Biggest Technical Challenge — Blocking

We have millions of records, so we **cannot** compare every Source 1 record against every Source 2 and Source 3 record — that would create an enormous number of comparisons.

Therefore our pipeline must first perform **Blocking / Candidate Generation**: instead of comparing against millions of records, we generate a small candidate set for each Source 1 entity.



The challenge specifically emphasizes that candidate generation needs to scale, and that the candidate set itself matters in the final evaluation.

6. What Features Can Help Us?

We should investigate multiple signals rather than relying on one field.

Business name

- Character similarity
- Token similarity
- N-gram similarity
- Spelling differences
- Abbreviation differences
- Extra/missing words

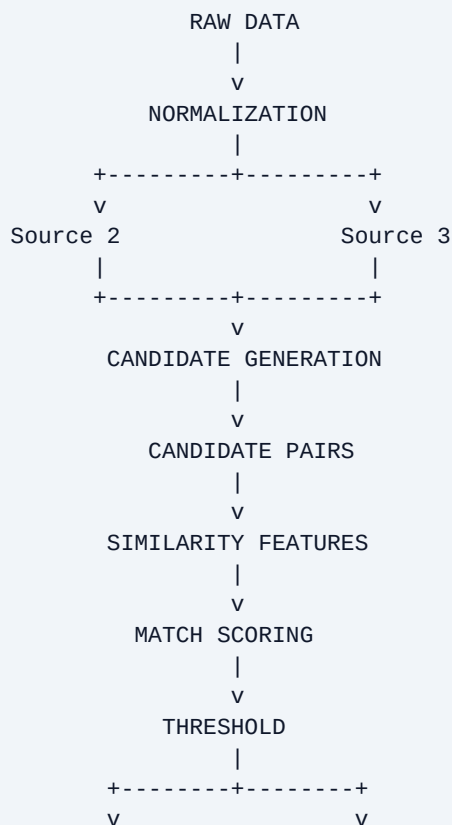
Address

- Token overlap
- Character similarity
- Normalized street names
- City/state similarity
- Number/address components

Country

Country can be useful for blocking/filtering and as a matching feature. For example, if Source 1 is in the US, a US candidate is generally more plausible than an Indian candidate — although this assumption should be validated against the training data rather than blindly enforced.

7. Proposed Overall Pipeline



matching_results.tsv candidate_pairs.tsv

8. Training vs Test

Training

We have Source 1, Source 2, Source 3, and Ground Truth. Ground truth tells us which Source 2/3 entities actually match each Source 1 entity. So we can use training data to:

- Test normalization
- Test blocking
- Create positive/negative candidate examples
- Build similarity features
- Train/calibrate the matching approach
- Evaluate $F_{1.5}$
- Tune thresholds

Test

For test data we have test_source1, test_source2 and test_source3 — but we don't have the answers. We'll run our final pipeline on the test data and generate the required submission files.

9. Evaluation

The challenge uses $F_{1.5}$. This means precision is weighted more heavily than recall, so we cannot simply generate thousands of possible matches and submit everything.

We need a balance:

- Too many candidates → low precision
- Too few candidates → lose true matches / low recall

That's why **both blocking quality and matching quality** matter.

10. Required Outputs

matching_results.tsv

Contains our predicted matches, conceptually:

source1_entity_id	matched_entity_ids
S1-xxxx	S2-xxxx, S3-xxxx, ...

candidate_pairs.tsv

Contains the candidates generated during blocking. This is especially important because the latest challenge update says candidate generation will also be considered in final ranking, not only the final matching score.

⇒ We should **not** treat candidate generation as an unimportant preprocessing step.

11. What We Should NOT Do

- ✗ Compare every Source 1 entity against every Source 2/3 entity.

- ✗ Use external business databases / API lookups.
- ✗ Use Google / Maps / business registries to resolve entities.
- ✗ Build a generic deep-learning model without first understanding the data.
- ✗ Create separate disconnected solutions that cannot be integrated.
- ✗ Push huge challenge datasets to public GitHub without confirming that this is permitted.

Our solution should be based on the challenge-provided data and permitted methods/resources.

■ 12. Team Responsibilities

Since we're 4 members:

Member 1 — Data + Normalization

Responsible for: data inspection, missing values, duplicate analysis, name normalization, address normalization, Unicode/multilingual handling.

Main module: `src/normalize.py`

Member 2 — Blocking / Candidate Generation

Responsible for: blocking strategies, candidate generation, candidate count, blocking recall, efficiency.

Main module: `src/blocking.py`

Important objective: reduce the search space while retaining as many true matches as possible.

Member 3 — Features + Matching

Responsible for: name similarity, address similarity, country features, token/character features, matching model/scoring, threshold experiments.

Main modules: `src/features.py`, `src/matching.py`

Me — Team Lead / Integration

I'll handle: overall pipeline, integration, validation, F₁ evaluation, experiments, submission generation, final documentation.

Main modules: `src/pipeline.py`, `src/validation.py`, `src/submission.py`

■ 13. Our Immediate Plan

We should **not** immediately run the complete 2.5 GB dataset through a model. We'll first do:

Phase 1 — Understand

- Dataset structure
- Match distribution
- Name/address patterns
- Missing values
- Multilingual cases

Phase 2 — Blocking experiment

- Use a manageable training sample and test Blocking Strategy A / B / C.
- Measure candidate count and how many known true matches were retained.

Phase 3 — Matching

- Build similarity features and a matching score.

Phase 4 — Validation

- Evaluate on held-out training data using F₁.

Phase 5 — Scale

- Only after the approach works: 1.7M test Source 1 + 4.8M Source 2 + 5.0M Source 3.

Phase 6 — Submission

- Generate matching_results.tsv and candidate_pairs.tsv, and validate them before submitting.
-

■ Most Important Thing for Everyone

Don't start coding random models yet.

First, Member 1 should understand normalization, Member 2 should investigate blocking, Member 3 should investigate similarity/matching, and the Team Lead will coordinate how these pieces fit together.

Our main goal is: generate a small, high-quality candidate set → accurately score the candidates → produce the required submission files efficiently. This is the problem we're actually solving — not simply "train an ML model."